

AI-BDM

Code Flow & Module Reference — for explaining the implementation, file by file, function by function.

This is the code-level companion to **AI-BDM_Architecture_And_Concepts_Guide.pdf** (which explains *what* the system does and the concepts behind it, for a non-technical reader). This document explains *how it's actually built* — which file calls which function, in what order, and why each module exists. Every call chain below was verified directly against the source code, not reconstructed from memory or documentation.

Read Section 1 first for the real, end-to-end story of what happens when the pipeline runs. Section 3 is the reference to come back to afterward, module by module.

Contents

- 1. The Complete Execution Trace — main.py, start to finish
- 2. The Two Entry Points (CLI vs. API) and the Async Job Layer
- 3. Module Reference — every file, grouped by responsibility
- 4. Tests — how the codebase proves itself correct
- 5. How to Summarize This in One Minute

1. The Complete Execution Trace

This is exactly what happens, in order, when someone runs:

```
python main.py --query "find 3 dental clinics in Islamabad with no online booking"
```

Notation: each line is one function call. Indentation shows nesting — an indented line is called *by* the line above it. This was traced directly from `main.py`'s `run()` function and the functions it calls into.

Entry point

main.py : main() — Parses the `--query/--concurrency/--no-rag/--no-maps` command-line arguments, then hands off to `run()`.

>> **main.py : run(query, concurrency, no_rag, no_maps)** — The actual orchestrator — everything below happens inside this one async function, in this order.

Steps 1 & 2 — Understand the request, then search and scrape

>> **main.py : _plan_interactively(query, concurrency)** — Wraps Step 1+2 so a clarification question can be answered in the same terminal session before continuing.

>> **phase1_pipeline.py : run_pipeline(query, ...)** — The real Stage 1+2 engine — everything from here down is inside this one call.

>> **phase1_pipeline.py : moderate_user_query(query)** — Safety check FIRST, before any paid API is touched. Blocks and returns immediately if the query violates usage policy.

>> **phase1_pipeline.py : deconstruct_intent(query)** — Understands the request.

>> **LLM_planner.py : plan_query(query)** — The actual LLM call (Groq) that turns the sentence into a structured plan: industry, location, count, intent, clarification decision, etc.

>> **model_router.py : select_model(TaskType.INTENT_PLANNING)** — Decides which Groq model/settings this specific call should use (see Section 3).

>> **phase1_pipeline.py : discover_targets(...) / discover_places(...)** — Finds candidate business URLs via Serper.dev (web search + Google Places).

>> **phase1_pipeline.py : process_single_lead(...)** — Runs once per candidate business — the biggest single function in the codebase. Expanded fully below.

>> **website_classifier.py : classify_homepage(...)** — Fast rule + LLM-assisted check: is this actually worth a deep crawl, or reject it now?

>> **route_filter.py : select_high_intent_routes(...)** — Picks which of a site's own links are worth crawling (contact/about/services pages), not every page on the site.

>> **discovery_classifier.py : classify_search_result(...)** — Distinguishes a real single business from a directory/listing page that should be mined for MORE businesses instead.

>> **relevance_scoring.py : score_relevance(...)** — Deterministic (non-LLM) scoring against the plan's industry/location/keywords — the main relevance gate.

>> **tech_stack.py : analyze_raw_html(domain, html, headers, cookies)** — Runs unconditionally on the homepage HTML already in memory (no extra fetch) — Stage 5's actual detection work, done here regardless of whether the query asked for it; only later SURFACING is conditional.

>> **phase3/review_harvester.py : enrich_business(...)** — Harvests customer reviews from whichever platforms make sense for this business's category (Google Reviews, Reddit, industry-specific sites).

>> **bi_providers.py : get_manager().discover_social_profiles(...)** — Finds the business's official social media profiles.

>> **github_enrichment.py : enrich_github(...)** / **youtube_enrichment.py : enrich_youtube(...)** — Two platforms with genuine, ToS-compliant official APIs — checked directly.

>> **bi_providers.py : get_manager().discover_decision_makers(...)** — Tries, in priority order: the site's own contact page, Schema.org structured data, LinkedIn (via Google search, never scraped directly), then general public search.

```
>> organization.py : build_organization(people) — Infers the business's likely org structure from whoever was found.  
>> business_intelligence.py : build_business_intelligence_summary(...) — Rolls all of the above into one consolidated per-business summary.  
>> storage.py : commit_domain(domain, ...) — Writes everything found for this business to storage/<domain>/ and adds its row(s) to crawl_index.csv — the actual save-to-disk step.  
>> storage_sync.py : sync_folder(domain, ...) — Optional — mirrors that folder to Cloudflare R2 if remote sync is configured.
```

Step 3 — Pick the right pages to focus on

```
>> route_planner.py : plan_routes(domain, query) — Runs once per discovered business.  
>> page_retrieval.py : hybrid_retrieve(...) — BM25 (keyword) + embedding (meaning) search over that business's own already-crawled pages — the common, no-LLM path.  
>> LLM_planner.py : (route-planning fallback call) — Only runs if hybrid_retrieve finds nothing usable — rare in practice.
```

Step 4 — Answer the specific question (only if the query asked one)

```
>> final_reasoning.py : answer_query(domain, query) — Runs once per business, only when the plan's intent is find_and_filter.  
>> (Groq LLM call, RAG pattern) — Reads the real page text Step 3 selected and answers strictly from it, citing sources, or says the evidence isn't there.
```

Step 5 — Surface tech-stack results (only if the query asked)

```
>> tech_stack.py : get_stored_profile(domain) — Reads back the profile that was ALREADY computed unconditionally during Step 2 (see analyze_raw_html above) — no new scan happens here unless a domain was somehow never profiled.
```

Cleaning — build the per-business rollup file

```
>> data_pipeline.py : run(routes, tech_stacks) — Reads crawl_index.csv (one row per page) and rolls it up into leads_clean.csv (one row per business).  
>> data_pipeline.py : to_business_level(...) — The actual per-page-to-per-business rollup logic — picks the best title/description, unions contact info across pages.
```

Step 6 — Build the searchable evidence index (skippable with --no-rag)

```
>> rag/ingest_and_answer.py : run(query, domains) —  
>> rag/chunker.py : chunk(...) — Breaks page/review text into small pieces.  
>> rag/embedder.py — Turns each chunk into a 384-number "meaning vector" (sentence-transformers, local, no API call).  
>> rag/store.py : get_store() — Saves the vectors into the local Chroma vector database (rag/chroma/).  
>> rag/retriever.py : retrieve(...) — Hybrid keyword + meaning search over the newly-built index, for this specific query.
```

Step 7 — Cross-check Google Maps (skippable with --no-maps)

```
>> phase3/google_maps.py : enrich("leads_clean.csv", "leads_with_maps.csv", geo) —  
>> phase3/google_maps.py : find_place(...) — Looks the business up via Serper's Google Places integration and matches it — address first, then phone, then name-only as a last resort — against the address already scraped from the business's own site.
```

Step 8 — Final quality check (always runs)

```
>> accuracy_check.py : run(...) — Read-only pass over everything just committed — no re-scraping.  
    >> accuracy_check.py : check_field_validity(...) / check_identity_consistency(...)  
      / check_review_relevance(...) — The three specific checks, written to accuracy_report.txt.
```

2. The Two Entry Points (CLI vs. API) and the Async Job Layer

Everything in Section 1 is reachable two ways: directly from a terminal, or over HTTP from another program.

Function	Reached via	What it does
main.py	CLI entry point	Runs the full 8-step trace above. This is the only path that reaches Steps 3–8 — the API (below) currently only reaches Steps 1–2.
api.py : run_pipeline_endpoint()	POST /api/v1/pipeline/run	Calls phase1_pipeline.run_pipeline() directly (Steps 1–2 only) and blocks until it's done. Supports an optional Idempotency-Key header so a retried call replays the original result instead of running twice.
api.py : submit_job() / job_status() / job_result() / cancel_job() / resume_job()	POST/GET /api/v1/jobs/*	The async alternative — submit returns immediately with a job_id; the caller polls for progress/status instead of blocking.
job_runner.py : submit_job() / _execute()	Called by api.py's submit_job()	Runs phase1_pipeline.run_pipeline() as a background task, feeding it progress-callback and cancel-check hooks, and turns the result into the versioned JobResult contract (job_contracts.py).
job_translation.py : target_to_query()	Called by job_runner._execute()	Converts a structured request (industry/location/company size) into the one natural-language sentence Stage 1's LLM_planner actually understands.

3. Module Reference

Every source file in the project, grouped by what it's responsible for.

Core pipeline (Stages 1–3)

File	Purpose	Key functions
main.py	CLI entry point; owns the full Step 1–8 orchestration shown in Section 1.	run(), main()
phase1_pipeline.py	The largest module — Stage 1+2's real engine: discovery, scraping, per-candidate processing, storage commit.	run_pipeline(), process_single_lead(), deconstruct_intent()
LLM_planner.py	Every LLM prompt/call for planning: the main plan, clarification questions, query-variation generation.	plan_query(), generate_query_variations()
model_router.py	The one place that decides which Groq model + settings a given task uses — nothing else hardcodes a model name.	select_model(), estimate_cost()
relevance_scoring.py	Deterministic (non-LLM) scoring of how well a candidate matches the plan's industry/location/keywords.	score_relevance()
discovery_classifier.py	Decides whether a search result is a real single business, a directory to mine, or noise to skip.	classify_search_result(), extract_businesses_from_html()
website_classifier.py	Decides whether a homepage is worth a full crawl at all, before spending more effort on it.	classify_homepage(), mine_homepage_businesses()
route_filter.py	Picks which links on a site are worth crawling (contact/about/services), filtering out junk.	select_high_intent_routes(), rank_urls()
route_planner.py	Stage 3 — picks the most relevant already-crawled pages for a specific query.	plan_routes()
page_retrieval.py	The hybrid keyword + meaning-based search route_planner uses in the common case.	hybrid_retrieve(), build_index()
page_patterns.py	Non-LLM homepage link pattern detection (e.g. finding a site's own nav structure).	extract_homepage_candidates(), detect_patterns()
page_intelligence.py	Generic, reusable per-page summary (page type, structure, content).	classify_page_type(), build_page_context()
final_reasoning.py	Stage 4 — answers the query's specific extra condition, grounded in real page text (RAG).	answer_query()
domain_utils.py	Tiny, dependency-light URL/domain helpers shared everywhere (kept separate to avoid pulling in heavy ML imports where not needed).	domain_key(), strip_tracking()
time_utils.py	One helper: consistent, Z-suffixed ISO-8601 timestamps everywhere a timestamp is generated.	utc_now_iso()
prompt_cache.py	Generic cache so an identical LLM prompt is never recomputed twice.	get(), set()

Technology detection (Stage 5)

File	Purpose	Key functions
tech_stack.py	Merges 4 independent detection engines (legacy Wappalyzer, extended cookie/DOM/JS, DNS, robots.txt) into one technology profile per business.	analyze_raw_html(), get_stored_profile(), build_website_profile()
vendor/wappalyzer/	A working, hand-patched copy of the Wappalyzer package, committed directly into the repo (not pip-installed) — see Section 8 of the Architecture guide for why.	(third-party, vendored)

Business intelligence / enrichment (part of Stage 2)

File	Purpose	Key functions
bi_providers.py	The pluggable manager that coordinates every enrichment source below into one consistent interface.	get_manager()
social_discovery.py	Finds a business's official social media links directly from its own site.	extract_social_links()
linkedin_discovery.py	Finds a company's LinkedIn page via Google search (never scrapes LinkedIn directly).	discover_company_linkedin(), discover_decision_makers()
linkedin_enrichment.py / linkedin_finder.py	Turns a confirmed LinkedIn match into structured company data.	enrich_linkedin_company(), find_linkedin()
github_enrichment.py / youtube_enrichment.py	Official-API-based enrichment for the two platforms with genuine, ToS-compliant APIs.	enrich_github(), enrich_youtube()
decision_maker_extractor.py	Decision-maker source #1: the business's own contact page.	extract_decision_makers(), extract_contact_page_people()
schema_org_extractor.py	Decision-maker source #2: structured data (Schema.org/JSON-LD) already embedded in the page.	extract_schema_org()
public_search_decision_makers.py	Decision-maker source #4 (last resort): general public search.	discover_via_public_search()
organization.py	Infers a plausible org structure from whichever decision-makers were actually found.	build_organization()
buying_committee.py	For each decision-maker found, infers who's actually likely to own a buying decision.	infer_likely_ownership(), annotate_buying_committee()
business_intelligence.py	Rolls every enrichment source above into one consolidated per-business summary.	build_business_intelligence_summary()
email_extractor.py	Production-grade email extraction/validation from scraped HTML.	domain_has_mx()

Reviews & Maps (Stage 2 harvesting + Stage 7)

File	Purpose	Key functions
phase3/review_harvester.py	Category-aware review discovery across whichever platforms make sense for a business's industry.	enrich_business(), platforms_for_category()
phase3/business_category.py	Classifies a business into a review-platform category (dental, marina, salon, etc.).	categorize_domain()
phase3/google_maps.py	Stage 7 — Google Places lookup and address/phone/name matching.	find_place(), enrich()
phase3/store.py	Phase 3's own tiny JSON cache, separate from the main storage layer.	load(), save()

File	Purpose	Key functions
phase3/config.py	Shared constants for the phase3 package.	(constants only)

Evidence index / RAG (Stage 6)

File	Purpose	Key functions
rag/chunker.py	Breaks page/review text into small, embeddable chunks.	chunk()
rag/embedder.py	Turns text chunks into meaning-vectors using the local sentence-transformers model.	(embedding function)
rag/store.py	Wraps the local Chroma vector database.	get_store()
rag/retriever.py	Hybrid keyword + meaning search over the vector store — the actual "R" in RAG.	retrieve()
rag/ingest_and_answer.py	Orchestrates chunk → embed → store → retrieve for a given query and set of businesses; what Step 6 actually calls.	run()
rag/ingest_from_storage.py / rag/ingest_from_csv.py / rag/ingest_reviews.py	Different entry points for feeding real scraped data into the index (from the storage/ folders, from a CSV, or reviews specifically).	iter_source_docs_from_storage(), etc.
rag/pipeline.py / rag/contract.py	Wires the RAG stages together and defines the data shapes flowing between them.	(orchestration / type definitions)
rag/top_matches.py / rag/ask.py / rag/eval.py / rag/generator.py / rag/llm_match_query.py	Standalone diagnostic/experimentation scripts — confirmed not imported by the real pipeline (rag/ingest_and_answer.py). Useful for manual debugging, not part of the production call chain.	(dev tools, not wired into main.py)

Storage & data rollout

File	Purpose	Key functions
storage.py	The main storage layer — writes/reads everything under storage/<domain>/ and crawl_index.csv.	get_store(), commit_domain()
storage_sync.py	Optional remote mirror of storage/ to Cloudflare R2.	get_provider(), sync_folder()
data_pipeline.py	Rolls crawl_index.csv (per-page) into leads_clean.csv (per-business); also the data-quality report tooling.	to_business_level(), run(), clean_rows()
accuracy_check.py	Stage 8 — the final read-only quality pass.	run(), check_field_validity(), check_identity_consistency(), check_review_relevance()

API & async job layer

File	Purpose	Key functions
api.py	FastAPI app — both the synchronous endpoint and the async job endpoints described in Section 2.	run_pipeline_endpoint(), submit_job(), job_status(), etc.
job_contracts.py	The versioned Pydantic request/progress/result/error data shapes for the async job system, plus the 8-class error taxonomy.	classify_error()
job_runner.py	The in-process job store and background execution engine for the async job system.	submit_job(), resume_job(), get_status(), cancel_job()
job_translation.py	Converts a structured job request into the natural-language query Stage 1 expects.	target_to_query()

Supporting / standalone tools

File	Purpose	Key functions
scripts/patch_wappalyzer.py	One-time tool for re-vendoring a newer Wappalyzer release in the future — not run during normal operation.	main(), verify()
async_scraper/	A separate, standalone scraping toolkit not called by main.py's pipeline at all — an independent utility living in the same repo.	(standalone package)

4. Tests — How the Codebase Proves Itself Correct

115+ automated tests, organized by how much of the real system each one touches:

Folder	What it actually proves
tests/smoke/	Do the core modules even import cleanly, and does the Stage 8 accuracy check work end-to-end against saved fixture data? No network, no API keys.
tests/unit/	Pure logic in isolation — request validation, scoring math, path handling — no I/O at all.
tests/mock/	Real logic, but with the network/provider calls replaced by fakes — proves the actual retry/caching/job/API behavior without needing real credentials or spending real money.
tests/contract/	Validates that example request/progress/result/error payloads match the versioned schemas exactly, and that the schema files haven't drifted from the code.
tests/integration/	The only tests that touch real, paid provider APIs (Groq, Serper, ScrapingBee, Apify) — deliberately excluded from the default test run, run only on request.

Full detail (142 hand-written test cases, mapped to exactly what's automated already) is in [AI-BDM_QA_Test_Plan.xlsx](#).

5. How to Summarize This in One Minute

If your manager asks "what did you actually build," here's the short version:

"I built the Python research engine for a B2B lead-generation system. It takes one plain-English request, uses an LLM to understand it, searches the web and scrapes real business websites, cross-checks each business against Google Maps and its own customer reviews, answers any specific question the request asked using a retrieval-augmented-generation pattern grounded in the real scraped text, and runs its own automated quality check before handing back results. It's exposed both as a command-line tool and as a REST API — including an async job system with progress tracking, cancellation, and resumability — so a separate Laravel backend can call it as part of a larger product. The whole thing has 115+ automated tests, is fully documented for handover, and runs in Docker alongside the rest of the system."

If asked to go one level deeper on any single piece, Section 3's module table gives you the exact file and function names to point to.